



Balloon Shooter Mini Game (in assembly)

Submitted To: Sikander Mehmood

Emaan Shakeel F24608001

Mataiba Rehman F24608007

Areeba Zeb F24608022

Zoya Rasool F24608056

Farkhunda Latif F24608060

Session: 2024-2028

Department of Computer Sciences SE

National University of Technology (NUTECH), Islamabad

Project Report

1. Executive Summary

Project Title: Balloon Shooter Game

Programming Language: x86 Assembly Language (MASM 6.15)

Libraries Used: Irvine32.lib

Platform: Windows 11

Project Type: Game

This report documents the Balloon Shooter game developed using x86 Assembly language with the Irvine32 library. The report focuses on the assembly language constructs, macros, constants, and programming techniques employed in the development.

2. Constants and Macros

2.1 Screen Dimension Constants

```
SCREEN_WIDTH = 80  
SCREEN_HEIGHT = 25
```

Purpose: Defines the game's playable area boundaries. These constants are resolved at assembly time and cannot be modified during execution.

2.2 Game Object Constants

```
MAX_BALLOONS = 7
```

Purpose: Controls the maximum number of active game objects and collision detection boundaries.

2.3 Color Constants

```
COLOR_BLACK = 0
COLOR_BLUE = 1
COLOR_GREEN = 2
COLOR_CYAN = 3
COLOR_RED = 4
COLOR_MAGENTA = 5
COLOR_BROWN = 6
COLOR_LIGHT_GRAY = 7
COLOR_DARK_GRAY = 8
COLOR_LIGHT_BLUE = 9
COLOR_LIGHT_GREEN = 10
COLOR_LIGHT_CYAN = 11
COLOR_LIGHT_RED = 12
COLOR_LIGHT_MAGENTA = 13
COLOR_YELLOW = 14
COLOR_WHITE = 15
```

Purpose: Provides named constants for the 16 standard VGA colors. Colors are packed as foreground and background.

2.4 Game State Constants

```
GAME_ACTIVE = 1
GAME_OVER = 2
```

Purpose: Enumerates possible game states for the main loop conditional branching.

2.5 Keyboard Scan Code Constants

```
VK_SPACE = 20h
VK_LEFT = 4Bh
VK_RIGHT = 4Dh
VK_ESC = 1
```

Purpose: Provides named constants for extended keyboard scan codes used in input processing.

2.6 Balloon Structure Offsets

```
BALLOON_X = 0
BALLOON_Y = 1
BALLOON_COLOR = 2
BALLOON_ACTIVE = 3
BALLOON_HIT = 4
BALLOON_DIR = 5
BALLOON_SIZE = 6
```

Purpose: Defines the memory layout for the balloon data structure. These offsets enable structure like access to array elements using base and displacement addressing.

3. Assembly Language Concepts Demonstrated

3.1 Data Types

Type	Size	Example	Usage
BYTE	8-bit	playerX BYTE 40	Coordinates, flags, colors
WORD	16-bit	Not used	none
DWORD	32-bit	score DWORD 0	Score, lives, timers
QWORD	64-bit	Not used	none

3.2 Addressing Modes

Immediate Addressing:

```
mov al, 40
```

Direct Addressing:

```
mov al, playerX
```

Register Indirect:

```
mov al, [esi]
```

Base+Displacement:

```
mov al, [esi + BALLOON_X]
```

3.3 Register Usage Map

Register	Purpose in Game
EAX	Return values, temporary storage, RandomRange output
EBX	Balloon X array pointer
ECX	Loop counter
EDX	Balloon color array pointer
ESI	Current balloon pointer (source index)
EDI	Balloon array base (destination index)

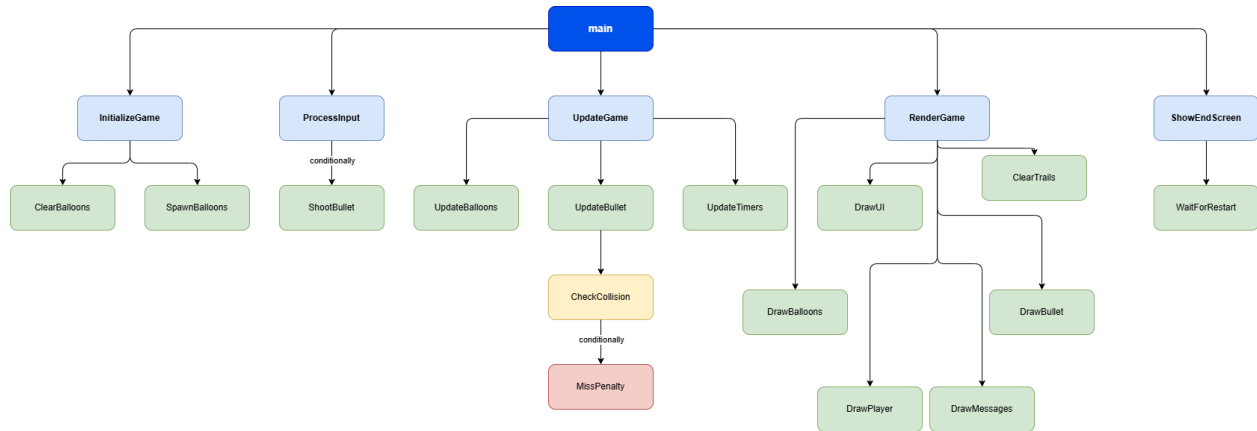
ESP	Stack pointer
-----	---------------

4. Complete Procedures Table

Procedure	Type	Purpose
main	Entry point	Game loop controller
InitializeGame	Setup	Resets all game variables
ClearBalloons	Utility	Deactivates all balloons
SpawnBalloons	Game logic	Creates new balloons
ProcessInput	Input handler	Keyboard event processing
ShootBullet	Action	Creates bullet object
UpdateGame	Game logic	Main update coordinator
UpdateBalloons	AI	Balloon movement logic
UpdateTimers	Utility	Message timer decrement
UpdateBullet	Physics	Bullet trajectory
MissPenalty	Game logic	Life deduction handler

CheckCollision	Detection	Hit testing
RenderGame	Graphics	Rendering coordinator
ClearTrails	Graphics	Previous frame cleanup
DrawUI	Graphics	Interface rendering
DrawBalloons	Graphics	Balloon rendering
DrawPlayer	Graphics	Player rendering
DrawBullet	Graphics	Bullet rendering
DrawMessages	Graphics	Popup message rendering
ShowEndScreen	UI	Game over display
WaitForRestart	Input	Restart/quit handler

4.2 Procedure Call Graph



5. Limitations and Constraints

Constraint	Reason
80×25 console required	Hardcoded screen dimensions
No mouse support	Keyboard only input
No sound effects	Irvine32 has no audio functions
Fixed frame rate	Delay based timing
No score persistence	No file I/O implementation

6. Conclusion

The Balloon Shooter game successfully demonstrates fundamental x86 Assembly language concepts including:

1. **Memory management** through direct addressing, structure offsets, and pointer arithmetic

2. **Register utilization** with efficient allocation of the 8 general-purpose registers
3. **Control flow** using conditional jumps, unconditional jumps, and the LOOP instruction
4. **Stack operations** for register preservation and parameter passing
5. **Interrupt handling** via Irvine32 library abstraction layer
6. **Real time execution** with frame-based timing using the Delay function
7. **Code organization** using procedures, segments, and labels
8. **Assembly directives** for data definition, constant declaration, and code structuring

The project serves as a practical example of Assembly language application in game development, demonstrating that real time interactive applications can be efficiently implemented in Assembly with minimal resource consumption. The use of macros, constants and self made procedures enhance code readability and maintainability while maintaining the low level control that Assembly provides.